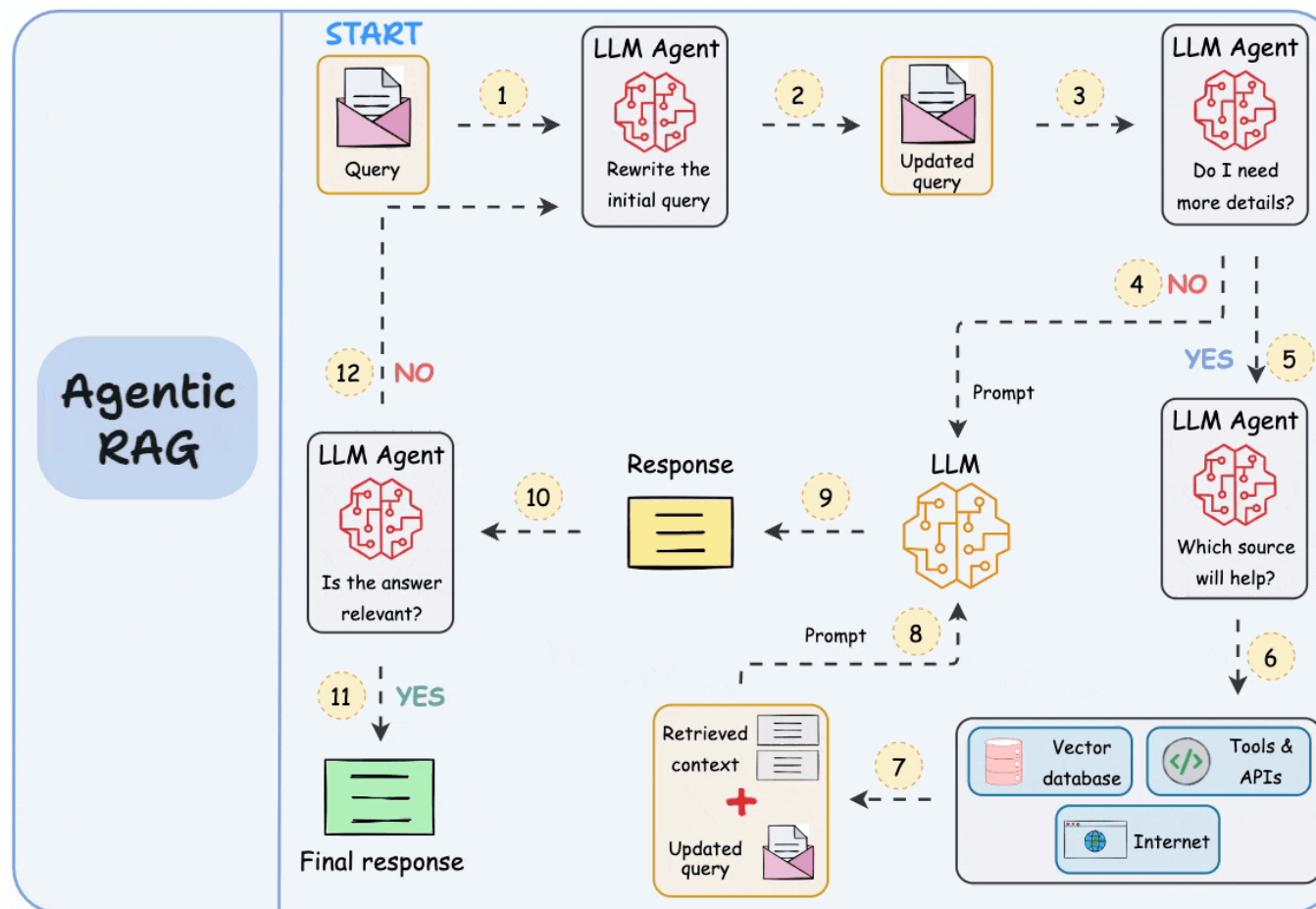


[< Go to the original](#)

Building Agentic RAG with LangGraph: Mastering Adaptive RAG for Production

**Piyush Agnihotri**

Follow

a11y-light · July 20, 2025 (Updated: July 21, 2025) · Free: No

Building Agentic Adaptive RAG with LangGraph for Production

If you're not a member yet, read for free [here](#).

Table of Contents:

· [Introduction](#) · [Adaptive RAG](#) · [Understanding the Adaptive RAG Workflow](#) · [What are we building?](#) · [Project Structure](#) · [Environment Setup with UV](#) · [Implementation Guide](#) · [Step 1: Define the State Management System & Essential Constants](#) · [Step 2: Define the chat model & the embedding model](#) · [Step 3: Build the Document Ingestion Pipeline](#) · [Step 4: Build the Query Router Chain](#) · [Step 5: Create the Document Retrieval Grader](#) · [Step 6: Build the Hallucination Detection System](#) · [Step 7: Create the Answer Quality Grader](#) · [Step 8: Build the Generation Chain](#) · [Step 9: Implement the Retrieval Node](#) · [Step 10: Create the Document Grading Node](#) · [Step 11: Build the Web Search Node](#) · [Step 12: Implement the Generation Node](#) · [Step 13: Construct the Complete Graph Workflow](#) · [Step 14:](#)

Introduction

Retrieval-Augmented Generation (RAG) has revolutionized how we build AI systems that can access and reason over external knowledge. However, as applications have grown more sophisticated, the limitations of traditional RAG approaches have become apparent. Today, we're witnessing an evolution from simple, linear RAG pipelines to intelligent, adaptive systems that can dynamically adjust their retrieval and generation strategies based on query complexity and context.

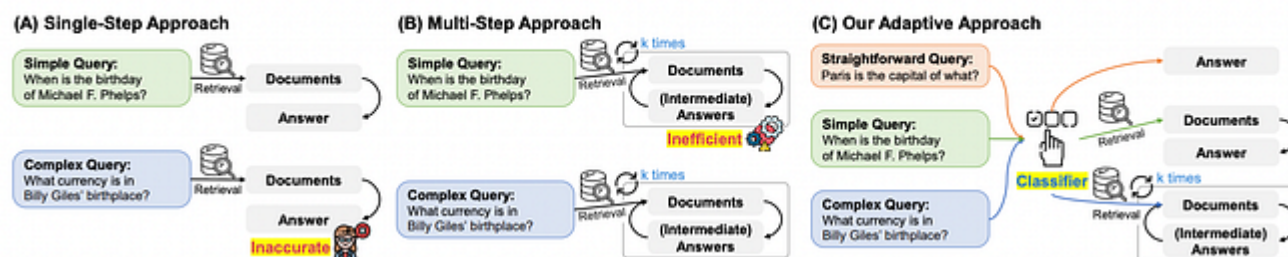
In this comprehensive guide, we'll explore **Agentic RAG** systems, specifically focusing on the implementation of Adaptive RAG using LangGraph and Google Gemini. But before we dive into the implementation, let's understand the Adaptive RAG.

Adaptive RAG

Adaptive RAG is an advanced strategy for RAG that intelligently combines (1) dynamic query analysis with (2) active/self-corrective mechanisms.

real-world queries exhibit vastly different complexity levels:

- **Simple queries:** "Paris is the capital of what?" — Can be answered directly by LLMs
- **Multi-hop queries:** "When did the people who captured Malakoff come to the region where Philipsburg is located?" — Requires four reasoning steps



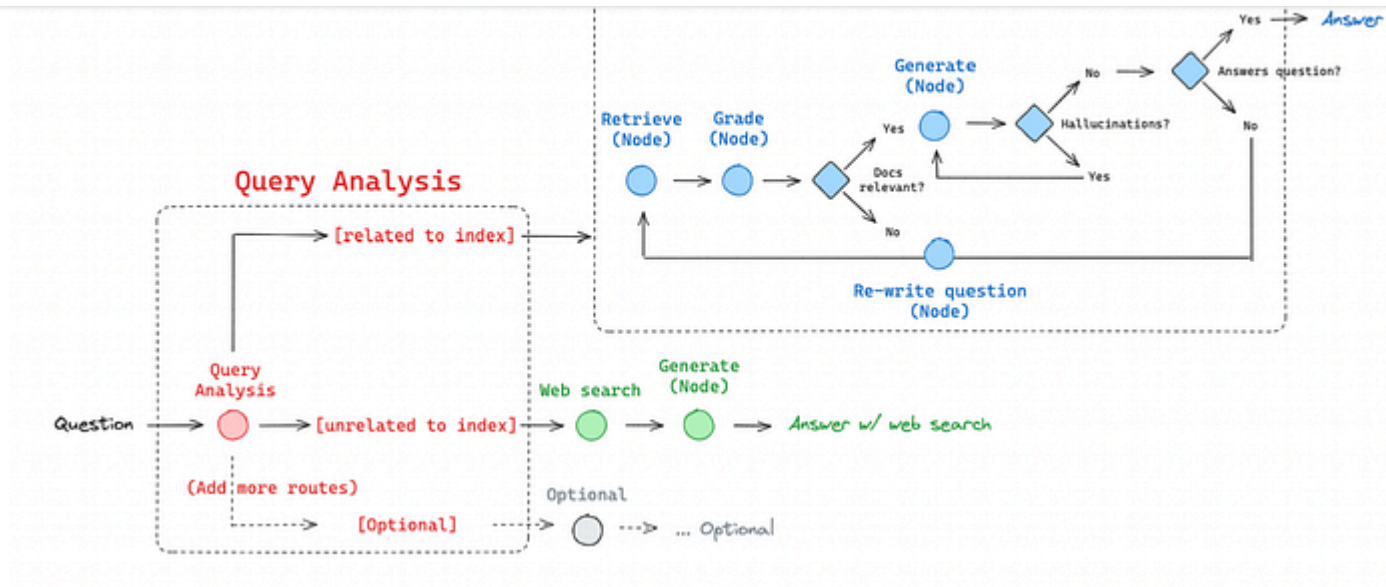
A conceptual comparison of different retrieval-augmented LLM approaches to question answering. (A) In response to a query, this single-step approach retrieves relevant documents and then generates an answer. However, it may not be sufficient for complex queries that require multi-step reasoning. (B) This multi-step approach iteratively retrieves documents and generates intermediate answers, which is powerful yet largely inefficient for the simple query since it requires multiple accesses to both LLMs and retrievers. (c) Our adaptive approach can select the most suitable strategy for retrieval-augmented LLMs, ranging from iterative, to single, to even no retrieval approaches, based on the complexity of given queries determined by our classifier. | **Credits:** [Soyeong Jeong et al., 2024](#)

The Core Problem: As illustrated in the comparative diagram:

- **(B) Multi-Step Approach:** Powerful for complex queries, but introduces unnecessary computational overhead for simple ones
- **(C) Adaptive Approach:** Uses a **query complexity classifier** to route each query to the most suitable strategy

Understanding the Adaptive RAG Workflow

Looking at the implementation diagram, we can see how Adaptive RAG orchestrates an intelligent decision-making process through a graph-based state management system:

Credits: langchain-ai

1. Query Routing & Classification

The system begins with a trained complexity classifier that analyzes the incoming question. This isn't just simple keyword matching; it's a sophisticated assessment that determines:

- Whether the query needs retrieval at all (parametric knowledge sufficient)
- If retrieval is needed, what level of complexity is required

2. Dynamic Knowledge Acquisition Strategy

Based on the complexity classification, the system intelligently routes between:

- **Index-based retrieval:** For queries answerable from the existing knowledge base
- **Web search:** For queries requiring fresh information or when local retrieval fails
- **No retrieval:** For queries answerable directly from the model's parametric knowledge

3. Multi-stage Quality Assurance

The system implements a comprehensive evaluation at multiple decision points:

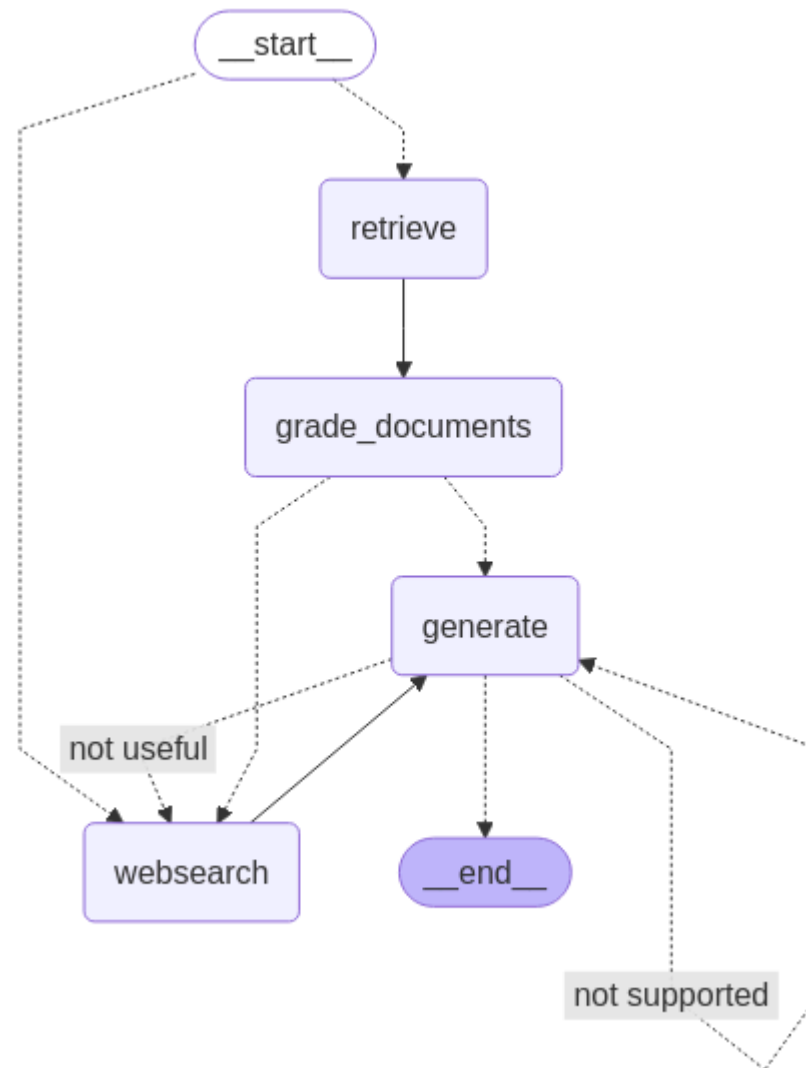
- **Document Relevance Assessment:** Uses confidence scoring to evaluate retrieval quality
- **Hallucination Detection:** Verifies generated answers are grounded in retrieved evidence

What are we building?

In this article, we explore the development of an advanced Retrieval-Augmented Generation (RAG) system capable of making intelligent decisions about information retrieval and routing. This implementation demonstrates sophisticated query analysis mechanisms that intelligently route user questions, establish robust evaluation frameworks to ensure response quality, and create adaptive architectures that seamlessly transition between multiple information sources.

This comprehensive guide presents a refactored approach to the original LangChain implementations, prioritizing enhanced code readability, improved maintainability, and superior developer experience. we'll implement an Adaptive RAG system using:

- **LangGraph** for orchestrating complex, stateful workflows
- **Google Gemini** as our primary language model
- **Vector databases** for efficient document retrieval
- **Web search integration** for real-time information access



Project Structure

Copy

```
building-adaptive-rag/
├── graph/
│   ├── chains/
│   │   ├── tests/
│   │   │   ├── __init__.py
│   │   │   └── test_chains.py
│   │   ├── __init__.py
│   │   ├── answer_grader.py
│   │   ├── generation.py
│   │   ├── hallucination_grader.py
│   │   ├── retrieval_grader.py
│   │   └── router.py
│   ├── nodes/
│   │   ├── __init__.py
│   │   ├── generate.py
│   │   ├── grade_documents.py
│   │   ├── retrieve.py
│   │   └── web_search.py
│   ├── __init__.py
│   ├── consts.py
│   ├── graph.py
│   └── state.py
├── static/
│   ├── LangChain-logo.png
│   └── Langgraph Adaptive Rag.png
```

```
├── .gitignore
├── ingestion.py
├── main.py
├── model.py
├── README.md
└── requirements.txt
```

This structure follows a clean, modular architecture where each component has a specific responsibility. The `graph` directory contains all the core logic, with `chains` handling LLM operations, `nodes` managing workflow steps, and the main `graph.py` orchestrating the entire system.

Environment Setup with UV

Let's start by setting up our development environment using UV, which is a fast Python package manager. First, make sure you have UV installed:

Copy

```
curl -LsSf <https://astral.sh/uv/install.sh> | sh
```

Now, let's create our project directory and set up the virtual environment:

```
mkdir building-adaptive-rag
cd building-adaptive-rag

uv venv --python 3.10
source .venv/bin/activate # On Windows: .venv\\Scripts\\activate
```

Create a `requirements.txt` file with all the necessary dependencies:

Copy

```
beautifulsoup4
langchain-community
tiktoken
langchainhub
langchain
langgraph
tavily-python
langchain-openai
python-dotenv
black
isort
pytest
langchain-chroma
langchain-tavily==0.1.5
langchain_aws
langchain_google_genai
```

Copy

```
uv pip install -r requirements.txt
```

Set up your environment variables by creating a `.env` file:

Copy

```
GOOGLE_API_KEY=your_tavily_api_key_here  
TAVILY_API_KEY=your_tavily_api_key_here # For web search capabilities  
LANGCHAIN_API_KEY=your_langchain_api_key_here # Optional, for tracing  
LANGCHAIN_TRACING_V2=true # Optional  
LANGCHAIN_ENDPOINT=https://api.smith.langchain.com # Optional  
LANGCHAIN_PROJECT=agentic-rag # Optional
```

Implementation Guide

Now let's build our Agentic RAG system step by step. I'll explain each component in the order that makes most sense for understanding the system flow.

Step 1: Define the State Management System & Essential Constants

We start with the foundation of our system — the state management. This is crucial because it defines how information flows through our graph.

Copy

```
from typing import List, TypedDict

class GraphState(TypedDict):
    """
    Represents the state of our graph.

    Attributes:
        question: question
        generation: LLM generation
        web_search: whether to add search
        documents: list of documents
    """

    question: str
    generation: str
    web_search: bool
    documents: List[str]
```

This `GraphState` class acts as the central data structure that flows through every node in our graph workflow. The `question` field holds the user's input query, `generation` stores the LLM's response, `web_search` is a boolean flag that determines whether we need to search the web for additional information, and `documents` contains all the retrieved documents from both local and web sources.

Create `graph/consts.py` :

Copy

```
RETRIEVE = "retrieve"  
GRADE_DOCUMENTS = "grade_documents"  
GENERATE = "generate"  
WEBSEARCH = "websearch"
```

These constants define the names of our graph nodes and help maintain consistency throughout our codebase. Having centralized constants makes it easier to refactor and reduces the risk of typos when referencing node names in our workflow definitions.

Step 2: Define the chat model & the embedding model

Create `model.py` :

Copy

```
# from langchain_openai import ChatOpenAI  
from langchain_aws import ChatBedrock  
# from langchain_openai import OpenAIEmbeddings
```

```
# llm_model = ChatOpenAI(temperature=0)
llm_model = ChatBedrock(model_id="us.anthropic.claude-sonnet-4-20250514-v1:0",

embed_model = GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")
```

Step 3: Build the Document Ingestion Pipeline

Create `ingestion.py` :

Copy

```
from dotenv import load_dotenv
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_chroma import Chroma
from langchain_community.document_loaders import WebBaseLoader
from model import embed_model

load_dotenv()

urls = [
    "https://lilianweng.github.io/posts/2023-06-23-agent/",
    "https://lilianweng.github.io/posts/2023-03-15-prompt-engineering/",
    "https://lilianweng.github.io/posts/2023-10-25-adv-attack-llm/",
]

docs = [WebBaseLoader(url).load() for url in urls]
docs_list = [item for sublist in docs for item in sublist]
```



```
,

doc_splits = text_splitter.split_documents(docs_list)

embed = embed_model

# Create vector store with documents
vectorstore = Chroma.from_documents(
    documents=doc_splits,
    collection_name="rag-chroma",
    embedding=embed,
    persist_directory="./.chroma",
)

# Create retriever
retriever = vectorstore.as_retriever()
```

This ingestion pipeline forms the backbone of our local knowledge base. We start by loading environment variables, then define a curated list of URLs containing high-quality content about AI agents, prompt engineering, and adversarial attacks.

The `WebBaseLoader` fetches content from these URLs and loads them into document objects. We then use the `RecursiveCharacterTextSplitter` to break down these documents into smaller, manageable chunks of 250 tokens each,

Finally, we create a Chroma vector store that will persist our embeddings locally, using *Google's text-embedding-004 model* for high-quality semantic representations.

Step 4: Build the Query Router Chain

Create `graph/chains/router.py` :

Copy

```
from typing import Literal

from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel, Field
from model import llm_model

class RouteQuery(BaseModel):
    """Route a user query to the most relevant datasource."""

    datasource: Literal["vectorstore", "websearch"] = Field(
        ...,
        description="Given a user question choose to route it to web search or"
    )

    llm = llm_model
```

```
system = """You are an expert at routing a user question to a vectorstore or web search. The vectorstore contains documents related to agents, prompt engineering, and adversarial attacks. Use the vectorstore for questions on these topics. For all else, use web-search"""

route_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system),
        ("human", "{question}"),
    ]
)

question_router = route_prompt | structured_llm_router
```

The query router is our system's first decision point that determines the optimal source for answering a user's question. We define a `RouteQuery` Pydantic model that constrains the router's output to either "vectorstore" or "websearch", ensuring reliable parsing of the LLM's decision.

The system prompt clearly defines the scope of our local knowledge base, instructing the router to use the vectorstore for questions about agents, prompt engineering, and adversarial attacks, while routing everything else to web search.

Step 5: Create the Document Retrieval Grader

Create `graph/chains/retrieval_grader.py` :

Copy

```
from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel, Field
from model import llm_model

llm = llm_model

class GradeDocuments(BaseModel):
    """Binary score for relevance check on retrieved documents."""

    binary_score: str = Field(
        description="Documents are relevant to the question, 'yes' or 'no'"
    )

structured_llm_grader = llm.with_structured_output(GradeDocuments)

system = """You are a grader assessing relevance of a retrieved document to a user question.
If the document contains keyword(s) or semantic meaning related to the question,
Give a binary score 'yes' or 'no' score to indicate whether the document is relevant"""

grade_prompt = ChatPromptTemplate.from_messages(
```



```
from pydantic import BaseModel, Field
from model import llm_model

llm = llm_model

class GradeHallucinations(BaseModel):
    """Binary score for hallucination present in generation answer."""

    binary_score: bool = Field(
        description="Answer is grounded in the facts, 'yes' or 'no'"
    )

structured_llm_grader = llm.with_structured_output(GradeHallucinations)

system = """You are a grader assessing whether an LLM generation is grounded in
    Give a binary score 'yes' or 'no'. 'Yes' means that the answer is grounded

hallucination_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system),
        ("human", "Set of facts: \n\n {documents} \n\n LLM generation: {generat
    ]
)

hallucination_grader: RunnableSequence = hallucination_prompt | structured_llm_
```

retrieved documents to verify that the information is factually grounded.

This prevents the system from generating plausible-sounding but factually incorrect responses. The grader uses a Boolean score to indicate whether the generation is supported by the provided facts.

When hallucinations are detected, our system can trigger regeneration or seek additional information, ensuring that users receive accurate and trustworthy responses.

Step 7: Create the Answer Quality Grader

Create `graph/chains/answer_grader.py` :

Copy

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableSequence
from pydantic import BaseModel, Field
from model import llm_model

class GradeAnswer(BaseModel):
```

```
,

llm = llm_model

structured_llm_grader = llm.with_structured_output(GradeAnswer)

system = """You are a grader assessing whether an answer addresses / resolves a
        Give a binary score 'yes' or 'no'. Yes' means that the answer resolves the
answer_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system),
        ("human", "User question: \n\n {question} \n\n LLM generation: {generation}")
    ]
)

answer_grader: RunnableSequence = answer_prompt | structured_llm_grader
```

The answer grader evaluates whether the generated response actually addresses the user's question. Even if a response is factually grounded, it might not directly answer what the user asked.

This component ensures that our system provides responses that are both accurate and relevant to the specific query. The grader checks if the generation resolves the question, and if not, the system can trigger additional retrieval or web search to find more appropriate information.

Create `graph/chains/generation.py` :

Copy

```
from langchain import hub
from langchain_core.output_parsers import StrOutputParser
from model import llm_model

llm = llm_model

prompt = hub.pull("rlm/rag-prompt")

generation_chain = prompt | llm | StrOutputParser()
```

The generation chain is responsible for creating the actual response to the user's question. We leverage a proven RAG prompt from LangChain Hub that has been optimized for retrieval-augmented generation tasks. This prompt template knows how to effectively combine retrieved context with the user's question to generate coherent, informative responses.

The chain uses `StrOutputParser` ensures we get clean string output that can be easily processed by subsequent components.

Create `graph/nodes/retrieve.py` :

Copy

```
from typing import Any, Dict

from graph.state import GraphState
from ingestion import retriever

def retrieve(state: GraphState) -> Dict[str, Any]:
    print("---RETRIEVE---")
    question = state["question"]

    documents = retriever.invoke(question)
    return {"documents": documents, "question": question}
```

The retrieve node is our first workflow step that fetches relevant documents from our local vector store. It takes the user's question from the state and uses our pre-configured retriever to find the most semantically similar documents.

The function returns both the retrieved documents and the original question, updating the state for the next nodes in the workflow. This node represents the

Step 10: Create the Document Grading Node

Create `graph/nodes/grade_documents.py` :

Copy

```
from typing import Any, Dict

from graph.chains.retrieval_grader import retrieval_grader
from graph.state import GraphState

def grade_documents(state: GraphState) -> Dict[str, Any]:
    """
    Determines whether the retrieved documents are relevant to the question
    If any document is not relevant, we will set a flag to run web search

    Args:
        state (dict): The current graph state

    Returns:
        state (dict): Filtered out irrelevant documents and updated web_search
    """

    print("----CHECK DOCUMENT RELEVANCE TO QUESTION----")
    question = state["question"]
    documents = state["documents"]
```

```

web_search = False
for d in documents:
    score = retrieval_grader.invoke(
        {"question": question, "document": d.page_content}
    )
    grade = score.binary_score
    if grade.lower() == "yes":
        print("---GRADE: DOCUMENT RELEVANT---")
        filtered_docs.append(d)
    else:
        print("---GRADE: DOCUMENT NOT RELEVANT---")
        web_search = True
from typing import Any, Dict

from graph.chains.retrieval_grader import retrieval_grader
from graph.state import GraphState

def grade_documents(state: GraphState) -> Dict[str, Any]:
    """
    Determines whether the retrieved documents are relevant to the question
    If any document is not relevant, we will set a flag to run web search

    Args:
        state (dict): The current graph state

    Returns:
        state (dict): Filtered out irrelevant documents and updated web_search
    """

```

```
documents = state["documents"]

filtered_docs = []
web_search = False
for d in documents:
    score = retrieval_grader.invoke(
        {"question": question, "document": d.page_content}
    )
    grade = score.binary_score
    if grade.lower() == "yes":
        print("---GRADE: DOCUMENT RELEVANT---")
        filtered_docs.append(d)
    else:
        print("---GRADE: DOCUMENT NOT RELEVANT---")
        web_search = True
        continue
return {"documents": filtered_docs, "question": question, "web_search": web_search}
```

The document grading node implements our quality control mechanism by evaluating each retrieved document for relevance to the user's question. It iterates through all retrieved documents and uses our retrieval grader to assess their relevance. Documents that are deemed relevant are added to the filtered list, while irrelevant documents are discarded.

external sources. This adaptive behavior ensures that users get comprehensive answers even when local knowledge is insufficient.

Step 11: Build the Web Search Node

Create `graph/nodes/web_search.py` :

Copy

```
from typing import Any, Dict
from dotenv import load_dotenv
from langchain.schema import Document
from langchain_tavily import TavilySearch
from graph.state import GraphState

load_dotenv()

web_search_tool = TavilySearch(max_results=3)

def web_search(state: GraphState) -> Dict[str, Any]:
    print("---WEB SEARCH---")
    question = state["question"]

    # Initialize documents - this was the missing part!
    documents = state.get("documents", []) # Get existing documents or empty

    tavily_results = web_search_tool.invoke({"query": question})["results"]
```

```
,
web_results = Document(page_content=joined_tavily_result)

# Add web results to existing documents (or create new list if documents was empty)
if documents:
    documents.append(web_results)
else:
    documents = [web_results]

return {"documents": documents, "question": question}

if __name__ == "__main__":
    web_search(state={"question": "agent memory", "documents": None})
```

The web search node extends our system's knowledge beyond the local vector store by querying the internet for current and comprehensive information. It uses Tavily, a search API optimized for AI applications, to find the most relevant web results.

The node retrieves up to 3 results, combines their content into a single document, and adds it to our document collection. This hybrid approach ensures that our system can handle both domain-specific queries using local knowledge and general queries requiring current information from the web.

Create graph/nodes/generate.py :

Copy

```
from typing import Any, Dict

from graph.chains.generation import generation_chain
from graph.state import GraphState

def generate(state: GraphState) -> Dict[str, Any]:
    print("---GENERATE---")
    question = state["question"]
    documents = state["documents"]

    generation = generation_chain.invoke({"context": documents, "question": question})
    return {"documents": documents, "question": question, "generation": generation}
```

The generation node is where our system creates the actual response to the user's question. It takes the user's question and all gathered documents (both from local retrieval and web search) and uses our generation chain to create a comprehensive answer. The node maintains all the context information in the state while adding the generated response, enabling subsequent nodes to evaluate the quality and accuracy of the generation.

Create graph/graph.py :

Copy

```
from dotenv import load_dotenv
from langgraph.graph import END, StateGraph

from graph.chains.answer_grader import answer_grader
from graph.chains.hallucination_grader import hallucination_grader
from graph.chains.router import RouteQuery, question_router
from graph.consts import GENERATE, GRADE_DOCUMENTS, RETRIEVE, WEBSEARCH
from graph.nodes import generate, grade_documents, retrieve, web_search
from graph.state import GraphState

load_dotenv()

def decide_to_generate(state):
    print("----ASSESS GRADED DOCUMENTS----")

    if state["web_search"]:
        print(
            "----DECISION: NOT ALL DOCUMENTS ARE NOT RELEVANT TO QUESTION, INCLUDE WEBSEARCH----"
        )
        return WEBSEARCH
    else:
        print("----DECISION: GENERATE----")
        return GENERATE
```

```

question = state["question"]
documents = state["documents"]
generation = state["generation"]

score = hallucination_grader.invoke(
    {"documents": documents, "generation": generation}
)

if hallucination_grade := score.binary_score:
    print("---DECISION: GENERATION IS GROUNDED IN DOCUMENTS---")
    print("---GRADE GENERATION vs QUESTION---")
    score = answer_grader.invoke({"question": question, "generation": generation})
    if answer_grade := score.binary_score:
        print("---DECISION: GENERATION ADDRESSES QUESTION---")
        return "useful"
    else:
        print("---DECISION: GENERATION DOES NOT ADDRESS QUESTION---")
        return "not useful"
else:
    print("---DECISION: GENERATION IS NOT GROUNDED IN DOCUMENTS, RE-TRY---")
    return "not supported"

def route_question(state: GraphState) -> str:
    """
    Route question to web search or RAG.

    Args:
        state (dict): The current graph state

```

```
    SET_NEXT_NODE_TO_CALL

"""

print("---ROUTE QUESTION---")
question = state["question"]
source: RouteQuery = question_router.invoke({"question": question})

if source.datasource == WEBSEARCH:
    print("---ROUTE QUESTION TO WEB SEARCH---")
    return WEBSEARCH
elif source.datasource == "vectorstore":
    print("---ROUTE QUESTION TO RAG---")
    return RETRIEVE

workflow = StateGraph(GraphState)

workflow.add_node(RETRIEVE, retrieve)
workflow.add_node(GRADE_DOCUMENTS, grade_documents)
workflow.add_node(GENERATE, generate)
workflow.add_node(WEBSEARCH, web_search)

workflow.set_conditional_entry_point(
    route_question,
    {
        WEBSEARCH: WEBSEARCH,
        RETRIEVE: RETRIEVE,
    },
)
```

```
workflow.add_edge(RETRIEVE, GRADE_DOCUMENTS)
workflow.add_conditional_edges(
    GRADE_DOCUMENTS,
    decide_to_generate,
    {
        WEBSEARCH: WEBSEARCH,
        GENERATE: GENERATE,
    },
)

workflow.add_conditional_edges(
    GENERATE,
    grade_generation_grounded_in_documents_and_question,
    {
        "not supported": GENERATE,
        "useful": END,
        "not useful": WEBSEARCH,
    },
)

workflow.add_edge(WEBSEARCH, GENERATE)
workflow.add_edge(GENERATE, END)

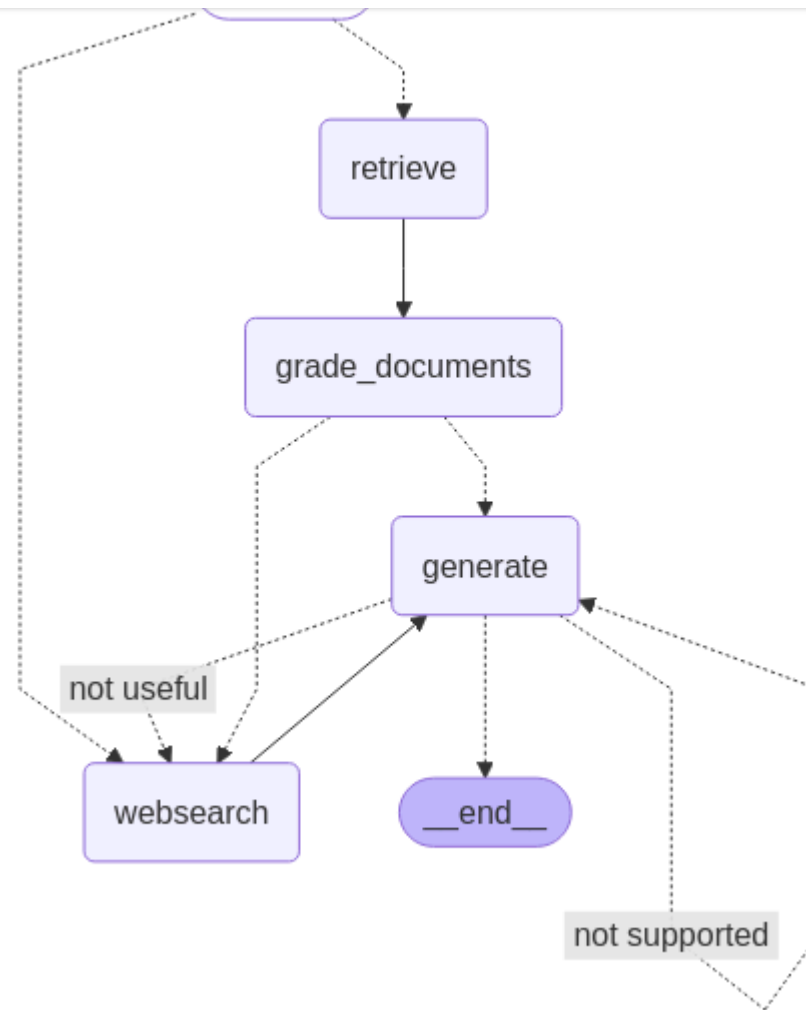
app = workflow.compile()

app.get_graph().draw_mermaid_png(output_file_path="graph.png")
```

- The graph starts with a conditional entry point that routes questions to either web search or local retrieval based on the router's decision.
- The workflow includes three key decision functions that implement our adaptive logic. The `decide_to_generate` function determines whether to proceed with generation or search the web based on document relevance scores.
- The `grade_generation_grounded_in_documents_and_question` function implements our self-correction mechanism by checking for hallucinations and answer relevance, potentially triggering regeneration or additional search.
- The `route_question` function handles initial query routing.

The compiled workflow creates a robust system that can adapt its behavior based on the quality of information at each step, ensuring reliable and comprehensive responses.

The final workflow will look like:



Step 14: Create the Main Application Entry Point

Create `main.py` :

```
from dotenv import load_dotenv

load_dotenv()

from graph.graph import app

def format_response(result):
    """Format the response from the graph for better readability"""
    if isinstance(result, dict) and "generation" in result:
        return result["generation"]
    elif isinstance(result, dict) and "answer" in result:
        return result["answer"]
    else:
        # Fallback to string representation
        return str(result)

def main():
    print("=" * 60)
    print("🤖 Advanced RAG Chatbot")
    print("=" * 60)
    print("Welcome! Ask me anything or type 'quit', 'exit', or 'bye' to stop.")
    print("-" * 60)

    while True:
        try:
            # Get user input
            user_question = input("\n💬 You: ").strip()
```

```
print("\n👋 Goodbye! Thanks for chatting!")
break

# Skip empty inputs
if not user_question:
    print("Please enter a question.")
    continue

# Show processing indicator
print("\n🤖 Bot: Thinking...")

# Process the question through the graph
result = app.invoke(input={"question": user_question})

# Format and display the response
response = format_response(result)
print(f"\n🤖 Bot: {response}")

except KeyboardInterrupt:
    print("\n👋 Goodbye! Thanks for chatting!")
    break
except Exception as e:
    print(f"\n❌ Sorry, I encountered an error: {str(e)}")
    print("Please try asking your question again.")

if __name__ == "__main__":
    main()
```


graph application, and demonstrates the working of the Advanced RAG Chatbot through the terminal.

Step 15: Comprehensive Testing Suite

Create `graph/chains/tests/test_chains.py` :

Copy

```
from pprint import pprint
import pytest

from dotenv import load_dotenv

load_dotenv()

from graph.chains.generation import generation_chain
from graph.chains.hallucination_grader import (GradeHallucinations,
                                                hallucination_grader)
from graph.chains.retrieval_grader import GradeDocuments, retrieval_grader
from graph.chains.router import RouteQuery, question_router
from ingestion import retriever

def test_retrival_grader_answer_yes() -> None:
    question = "agent memory"
```

```
doc_txt = docs[1].page_content

res: GradeDocuments = retrieval_grader.invoke(
    {"question": question, "document": doc_txt}
)

assert res.binary_score == "yes"

def test_retrival_grader_answer_no() -> None:
    question = "agent memory"
    docs = retriever.invoke(question)

    # Skip test if no documents are returned
    # if len(docs) == 0:
    #     pytest.skip(f"No documents returned for query: {question}")

    doc_txt = docs[1].page_content

    res: GradeDocuments = retrieval_grader.invoke(
        {"question": "how to make pizaa", "document": doc_txt}
    )

    assert res.binary_score == "no"

def test_generation_chain() -> None:
    question = "agent memory"
    docs = retriever.invoke(question)
```

```
def test_hallucination_grader_answer_yes() -> None:
    question = "agent memory"
    docs = retriever.invoke(question)

    generation = generation_chain.invoke({"context": docs, "question": question})
    res: GradeHallucinations = hallucination_grader.invoke(
        {"documents": docs, "generation": generation}
    )
    assert res.binary_score

def test_hallucination_grader_answer_no() -> None:
    question = "agent memory"
    docs = retriever.invoke(question)

    res: GradeHallucinations = hallucination_grader.invoke(
        {
            "documents": docs,
            "generation": "In order to make pizza we need to first start with t"
        }
    )
    assert not res.binary_score

def test_router_to_vectorstore() -> None:
    question = "agent memory"
```

```
def test_router_to_websearch() -> None:
    question = "how to make pizza"

    res: RouteQuery = question_router.invoke({"question": question})
    assert res.datasource == "websearch"
```

This comprehensive test suite validates each component of our Agentic RAG system independently.

- The tests cover document relevance grading with both positive and negative cases, ensuring our grader correctly identifies relevant and irrelevant documents.
- The hallucination detection tests verify that our system can distinguish between grounded and fabricated responses.
- The router tests confirm that questions are correctly routed to vectorstore or web search based on their content.

These tests will ensure that each component works correctly in isolation before integration into the complete workflow.

Running the System

To run your Agentic RAG system:

1. First, create the vector database:

Copy

```
python ingestion.py
```

This code creates a vector store that persists to a `.chroma` directory by downloading documents from specified URLs, splitting them into smaller chunks, generating embeddings for each chunk, and storing all of this processed data in a Chroma database for later retrieval and similarity searching.

Check if the Chroma vector store has been created or not.

Copy

```
ls -la .chroma
```

Expected response:

```
total 2004
drwxrwxr-x 3 ubuntu ubuntu    4096 Jul 20 14:56 .
drwxr-xr-x 9 ubuntu ubuntu    4096 Jul 20 14:56 ..
-rw-r--r-- 1 ubuntu ubuntu 3022848 Jul 20 14:56 chroma.sqlite3
drwxrwxr-x 2 ubuntu ubuntu    4096 Jul 20 14:56 d065d653-04c5-48de-a4d9-cd33b57
```

Run the main application:

Copy

```
python main.py
```

Expected Interface:

Copy

```
📖 Loading existing vector store...
=====
🤖 Advanced RAG Chatbot
=====
Welcome! Ask me anything or type 'quit', 'exit', or 'bye' to stop.
-----

💬 You: what is agent memory?

🤖 Bot: Thinking...
```

```
---RETRIEVE---
```

```
---CHECK DOCUMENT RELEVANCE TO QUESTION---
```

```
---GRADE: DOCUMENT RELEVANT---
```

```
---GRADE: DOCUMENT RELEVANT---
```

```
---GRADE: DOCUMENT RELEVANT---
```

```
---GRADE: DOCUMENT RELEVANT---
```

```
---ASSESS GRADED DOCUMENTS---
```

```
---DECISION: GENERATE---
```

```
---GENERATE---
```

```
---CHECK HALLUCINATIONS---
```

```
---DECISION: GENERATION IS GROUNDED IN DOCUMENTS---
```

```
---GRADE GENERATION vs QUESTION---
```

```
---DECISION: GENERATION ADDRESSES QUESTION---
```

🤖 Bot: Agent memory **is** a key component **of** LLM-powered autonomous agent system.

💬 You: quit

👋 Goodbye! Thanks **for** chatting!

Execute the test suite:

Copy

```
python -m pytest . -s -v
```

Expected response:

```

===== test session =====
platform linux -- Python 3.10.12, pytest-8.4.1, pluggy-1.6.0 -- /home/ubuntu/anaconda3/bin/python
cachedir: .pytest_cache
rootdir: /home/ubuntu/github/langgraph-ai/agentrag/agentrag-systems/build
plugins: anyio-4.4.0, langsmith-0.3.45
collecting ... 📖 Loading existing vector store...
collected 7 items

graph/chains/tests/test_chains.py::test_retrival_grader_answer_yes PASSED
graph/chains/tests/test_chains.py::test_retrival_grader_answer_no PASSED
graph/chains/tests/test_chains.py::test_generation_chain ('Agent memory in LLM-
'sensory memory (learning embedding representations for raw inputs), '
'short-term memory (in-context learning limited by the model's context '
'window), and long-term memory (external vector stores accessible via fast '
'retrieval). Short-term memory utilizes the model's finite context capacity, '
'while long-term memory provides the capability to retain and recall '
'information over extended periods by leveraging external databases. The '
'memory system also includes retrieval mechanisms that surface relevant '
'context based on recency, importance, and relevance to guide the agent's '
'behavior.')
PASSED
graph/chains/tests/test_chains.py::test_hallucination_grader_answer_yes PASSED
graph/chains/tests/test_chains.py::test_hallucination_grader_answer_no PASSED
graph/chains/tests/test_chains.py::test_router_to_vectorstore PASSED
graph/chains/tests/test_chains.py::test_router_to_websearch PASSED

===== 7 passed, 2 warnings =====

```

Conclusion

Building an Agentic RAG system with LangGraph represents a significant advancement over traditional RAG approaches. By incorporating intelligent decision-making, self-reflection, and adaptive strategies, we create systems that can reason about information quality, adapt retrieval strategies, and self-correct when necessary.

This implementation provides a solid foundation for production-ready applications that require reliable, accurate, and intelligent information retrieval and generation. The modular design allows for easy extension and customization based on specific use cases and requirements.

Limitations/Next Steps

To further enhance this system, consider:

- **LLM Fallback State:** Add an `llm_fallback` state to handle queries unrelated to both websearch and vectorstore content. For instance, when a user says "Hey" or asks general conversational questions, the system should route to a direct LLM response rather than attempting retrieval or web search.

guidance on when to use each of the three routes: **Vectorstore** (for document-specific queries), **Websearch** (for current information needs), and **LLM Fallback** (for general conversation, greetings, or queries outside the scope of stored documents and web search)

You can find the complete source code for this blog in the repository below. [Github Repo](#)

Acknowledgement: [Mistralai](#) and [langchain](#)

Thank you for reading!

I hope you found something valuable. If you find this post valuable:

- Show your support with a 🖐️ **clap (or many!)**
- Share it with fellow AI or Python enthusiasts who might benefit
- Leave a comment — your feedback keeps this series going strong

Let's keep learning, building, and exploring the power of AI together.

You can connect with me on [LinkedIn](#), [Github](#) & [Instagram](#)

[#artificial-intelligence](#) [#genai](#) [#machine-learning](#) [#agentic-ai](#) [#langgraph](#)

Freedium

[ko-fi](#) [librepat](#) [patreon](#)
